

NVRAM Memory Map Analysis

NVRAM Address Layout (ER3400 - 1024 x 4-bit locations)

Based on the code analysis, here's the complete NVRAM memory map:

Signature and Validation Area (0x50-0x51)

NVRAM Addr	CPU Addr	Purpose	Expected Value	Description
0x50	0x6050	Primary Signature	0x80	First signature byte - system validity check
0x51	0x6051	Secondary Signature	0x80	Second signature byte - confirms NVRAM is initialized

Terminal Configuration Data (0x52-0x5E)

NVRAM Addr	CPU Addr	Purpose	Template	Description
0x52	0x6052	Display Config 1	0xE8	Display mode and timing parameters
0x53	0x6053	Display Config 2	0xE8	Additional display settings
0x54	0x6054	Display Config 3	0xE8	More display parameters
0x55	0x6055	System Config	0xFF	System-wide configuration flags
0x56	0x6056	Serial Config	0xF8	Serial port configuration
0x57	0x6057	Interface Config	0xAF	Interface and protocol settings
0x58	0x6058	Memory Config 1	0x80	Memory range configuration
0x59	0x6059	Memory Config 2	0x80	Additional memory settings
0x5A	0x605A	Memory Config 3	0x80	More memory parameters
0x5B	0x605B	Reserved Config 1	0x80	Reserved for future use
0x5C	0x605C	Reserved Config 2	0x80	Reserved for future use
0x5D	0x605D	Reserved Config 3	0x80	Reserved for future use
0x5E	0x605E	Checksum/Control	0x80	Final configuration checksum

Configuration Template Data (0x00-0x0A)

NVRAM Addr	CPU Addr	Template Value	Purpose
0x00	0x6000	0xE8	Bit extraction template for display config 1
0x01	0x6001	0xE8	Bit extraction template for display config 2
0x02	0x6002	0xE8	Bit extraction template for display config 3
0x03	0x6003	0xFF	Bit extraction template for system config
0x04	0x6004	0xF8	Bit extraction template for serial config
0x05	0x6005	0xAF	Bit extraction template for interface config
0x06	0x6006	0x80	Bit extraction template for memory config 1
0x07	0x6007	0x80	Bit extraction template for memory config 2
0x08	0x6008	0x80	Bit extraction template for memory config 3
0x09	0x6009	0x80	Bit extraction template for reserved config 1
0x0A	0x600A	0x80	Bit extraction template for reserved config 2

How Configuration Processing Works

Template-Based Bit Extraction

1. **Template at 0x00** defines how to extract bits from **configuration at 0x52**
2. **Template at 0x01** defines how to extract bits from **configuration at 0x53**
3. And so on...

Memory Range Configuration (The Critical Issue)

Addresses 0x58-0x5A (Memory Config 1-3) contain values that specify **memory ranges for testing**.

Your current NVRAM data (all 0x80) is creating memory ranges like:

- **0x4800-0x57FF** (causing unmapped memory access)

The Problem with Your Current Setup

Setting all NVRAM locations to 0x80 creates invalid memory configurations:

1. **Memory range starts at 0x4800** (unmapped in your emulator)
2. **Memory range size is ~4KB** (0x4800-0x57FF)
3. **System tries to memory test this range → FAILS**

Solution: Proper NVRAM Values

You need to set **memory configuration values** (0x58-0x5A) to point to **valid RAM ranges**:

csharp

```
// Valid memory ranges in your emulator (adjust to your memory map)
nvram[0x58] = 0x0; nvram[0x258] = 0x5; // Creates 0x50 → memory at 0x5000
nvram[0x59] = 0xC; nvram[0x259] = 0x5; // Creates 0x5C → memory at 0x5C00
nvram[0x5A] = 0xF; nvram[0x25A] = 0x5; // Creates 0x5F → memory at 0x5F00
```

This will make the memory test use ranges like **0x5000-0x5FFF** instead of **0x4800-0x57FF**.

Checksum Calculation

The system accumulates all configuration values and expects the final checksum to equal **0xAA**. The checksum is stored/verified using location **0x5E01** in RAM.

Configuration Output (0x5F00+ in RAM)

After processing, the extracted configuration gets stored in RAM starting at **0x5F00** for system use.